PACE Solver Description: Woodcutter, a Branch-Cut-and-Price Solver for Maximum Agreement Forest

Johannes Varga
Algorithms and Complexity Group, TU Wien, Austria

Enrico Iurlano
Algorithms and Complexity Group, TU Wien, Austria

Hai Xia
Algorithms and Complexity Group, TU Wien, Austria

Abstract

We describe `woodcutter`, our solver for the *Parameterized Algorithms and Computational Experiments* (PACE) challenge of 2026 involving the (rooted and binary) *Maximum Agreement Forest* (MAF) problem. The solver's core is an exact branch-cut-and-price procedure over a set-packing formulation whose columns are agreement subtrees. Columns are generated by solving a weighted variant of the Maximum Agreement Subtree problem on the internal vertices of the input trees; the linear programming bound is strengthened by clique cuts and closed by branching. This branch-cut-and-price procedure is preceded by reduction rules and a cluster decomposition. The heuristic and lower-bound version of the solver heavily rely on the same machinery: the heuristic version drives a column-pool matheuristic to a valid incumbent under a time budget, and the lower-bound version uses the converged column-generation dual, which is a valid lower bound on the optimum.

2012 ACM Subject Classification Mathematics of computing → Combinatorial optimization; Theory of computation → Branch-and-bound

Keywords and phrases Maximum Agreement Forest, Phylogenetics, Branch-and-Price, Column Generation, PACE Challenge

Digital Object Identifier 10.4230/LIPIcs...

Supplementary Material *Software (source code)*: `https://github.com/jovarga/woodcutter`

1 Introduction

Let $X = \{1, \ldots, n\}$ be a set of *taxa* and let $T_1, \ldots, T_t$ be rooted binary trees, each having leaf set X (each T_i is a so-called phylogenetic tree). For a rooted binary tree T with leaf set X and a subset $S \subseteq X$, let $T|S$ denote the minimal rooted subtree of T connecting the leaves in S with all resulting outdegree-one vertices contracted, and let $\mathrm{St}(T, S)$ denote the set of internal vertices of T that lie on this minimal subtree (its Steiner tree). An *agreement forest* is a partition $\mathcal{F} = \{S_1, \ldots, S_k\}$ of X such that

(i) for every block S_j the restricted trees $T_1|S_j, \ldots, T_t|S_j$ are isomorphic, and

(ii) for every input tree T_i the Steiner trees $\mathrm{St}(T_i, S_1), \ldots, \mathrm{St}(T_i, S_k)$ are pairwise vertex-disjoint.

The *Maximum Agreement Forest* problem asks for an agreement forest $\mathcal{F}$ with the minimum number of blocks $k^* := |\mathcal{F}|$. Our approach attacks the problem with a branch-cut-and-price (BCP) algorithm over an Integer Linear Programming (ILP) formulation with one binary decision variable per *agreement subtree*: a taxon subset on which all input trees induce a common rooted topology (equivalently, a *valid block*; Section 2). `woodcutter` is implemented in C++ and solves all its linear and mixed-integer programs with the HiGHS solver [10].

The remainder of this description is structured as follows. Section 2 presents the set-packing ILP formulation of MAF and notes that its linear programming (LP) relaxation is solved by column generation. Section 3 gives an overview of the exact branch-cut-and-price solver together with its reduction and cluster-decomposition wrapper, and the subsequent sections detail its four ingredients: column generation (Section 4), branching (Section 5), cutting planes (Section 6), and the pricing subproblem solver (Section 7). Sections 8 and 9 finally describe how the same machinery is adapted to the heuristic and the lower-bound track, respectively.

2 ILP Formulation

Let $\mathcal{S}$ be the set of all *valid blocks*: subsets $S \subseteq X$ with $|S| \geq 2$ such that $T_1|S, \ldots, T_t|S$ are isomorphic. We introduce a binary variable a_S for every $S \in \mathcal{S}$ and model MAF as a set-packing program on the Steiner-vertex incidence:

$$\min \quad n - \sum_{S \in \mathcal{S}} (|S| - 1)\, a_S \tag{1}$$

$$\text{s.t.} \quad \sum_{S \in \mathcal{S} \,:\, v \in \mathrm{St}(T_i, S)} a_S \leq 1, \qquad i = 1, \ldots, t,\ v \in \mathrm{St}(T_i, X). \tag{2}$$

$$a_S \in \{0, 1\}, \qquad S \in \mathcal{S}. \tag{3}$$

Every taxon left uncovered by the selected blocks is considered emitted as singleton block; singletons have an empty Steiner tree and are therefore trivially valid and disjoint from every other block. Starting from the all-singletons forest of n blocks, selecting a block S merges $|S|$ singletons into one block and thus saves $|S| - 1$ blocks, which explains the objective (1). The ILP (1)–(3) is a simplification of the ILP that has been proposed by Olver et al. [14] and computationally examined by Frohn et al. [9].

A sketch of correctness of the formulation. Any $\{0, 1\}$-vector satisfying (2), completed by singletons, is an agreement forest: each selected S is valid by definition, and (2) makes the Steiner trees of the selected blocks pairwise vertex-disjoint in every tree. Vertex-disjointness also forces *leaf*-disjointness of the non-singleton blocks: if two blocks $S \neq S'$ with $|S|, |S'| \geq 2$ shared a taxon x, then in each tree the parent of x would lie on both Steiner trees, violating (2). Hence the selection plus singleton completion is a partition, i.e. an agreement forest, and its block count equals (1). Conversely every agreement forest yields a feasible selection of its non-singleton blocks.

Thus the optimum of the ILP (1)–(3) is k^*, and the value of its Linear Programming (LP) relaxation is a valid lower bound on k^*. The family $\mathcal{S}$ is exponential, so the LP relaxation is solved by column generation [2, 13], and integrality is enforced by branch-and-price.

3 Exact solver: Branch-Cut-and-Price

The exact solver combines column generation, cutting planes, and branching into a single branch-cut-and-price loop, wrapped by the reduction rules and the cluster decomposition; Figure 1 summarizes the resulting pipeline. It maintains a *restricted master problem* (RMP): the LP relaxation of (1)–(2) over a growing subset of the columns of the full *master problem* (MP), held in a single persistent, warm-started HiGHS model to which columns and rows are only ever appended. At the root, column generation is iterated to LP optimality, optionally interleaved with clique-cut separation; a mixed-integer solve over the columns collected so far provides a primal incumbent. If the integrality gap is below one, the incumbent is optimal.

Otherwise the solver branches, producing a best-bound branch-and-price tree in which each node owns a clone of the master (preserving all row and column indices so cut and branch rows stay valid) and re-runs column generation under its branching constraints. Figure 1 shows the whole exact pipeline; the following four subsections describe the four ingredients of this loop, and Section 3 assembles them with the reduction/decomposition wrapper.

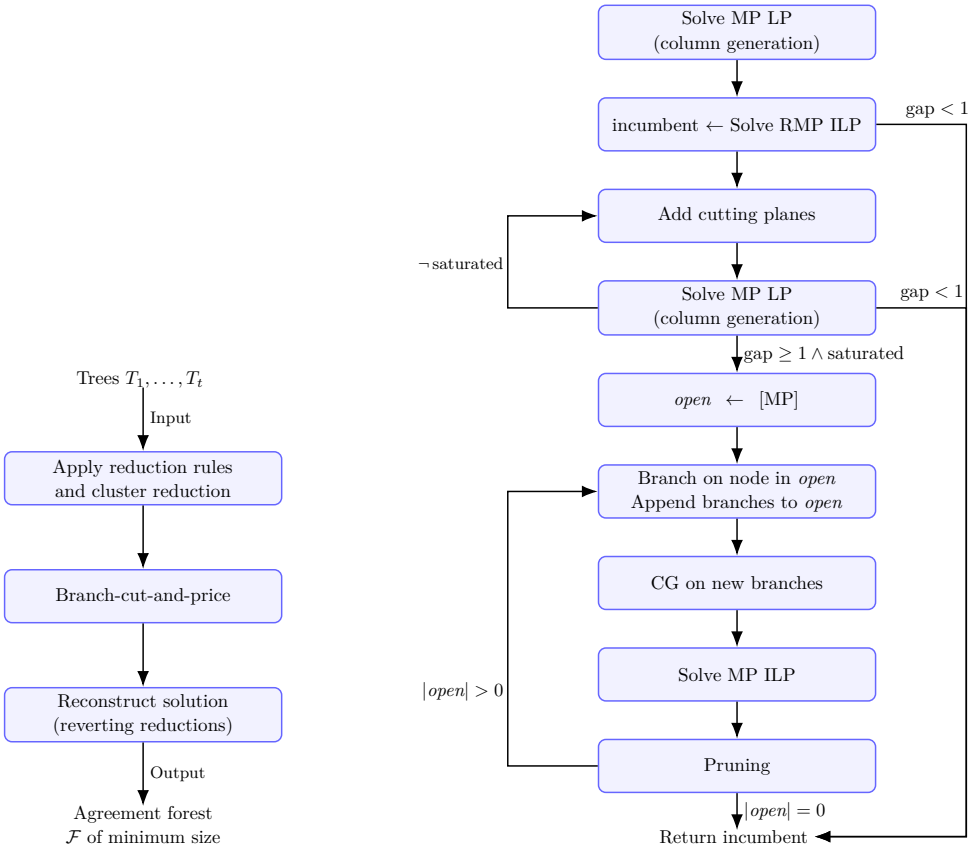

Figure 1 The exact pipeline (Section 3). *Left:* the top-level flow—the input trees (T_i) are simplified by the reduction rules and the cluster reduction, each irreducible instance is solved by branch-cut-and-price, and the reductions are reverted to reconstruct an agreement forest $\mathcal{F}$ of minimum size. *Right:* a single branch-cut-and-price call. The LP relaxation of the master problem (MP) is solved by column generation (CG; Section 4); a mixed-integer solve over the columns collected in the restricted master (RMP) yields a primal incumbent; clique cutting planes (Section 6) are then separated and the LP re-optimized until no violated clique remains (*saturated*). If the integrality gap drops below one the incumbent is optimal and is returned; otherwise best-bound branch-and-bound (Section 5) begins, and each open node re-runs column generation under its branching constraints. Every "Solve MP LP" box is itself closed by the pricing subproblem of Section 7, so that column generation runs under the hood of every node of the branch-and-bound tree. Abbreviations: MP, master problem; RMP, restricted master problem; LP, linear program; ILP, integer linear program; CG, column generation. Gap is the primal bound minus the dual bound.

Reduction rules.

Before search we exhaustively apply, in a fixed order, the standard kernelization rules for agreement forests, each of which we use as stated in the literature (noting the deviations below):

- *Common cherry / pendant-subtree reduction* [3]: if two leaves are siblings (a cherry) in every input tree, they must share a block, so one is absorbed into the other. Iterating this over binary trees contracts every common pendant subtree; it is valid for any t and does not change k^*.
- *Chain reduction*: a maximal chain of pendant leaves common to both trees can be truncated [12]. We use the $(4, 3)$ variant of the improved linear kernel [11] (a common chain of length four is shortened to length three by absorbing the fourth link) and apply it only for $t = 2$; it does not change k^*.
- $(3, 2)$ *near-cherry rule* [11]: a specific near-cherry configuration forces one leaf into its own singleton block; the leaf is removed and the optimum is incremented by one. We apply this rule only for $t = 2$.

Each firing is recorded on an undo stack so the reduced-instance forest can be expanded back to the original labels exactly. For $t \geq 3$ only the common-cherry rule fires.

Cluster reduction.

We then decompose the instance into its common clusters using the Linz–Semple cluster reduction [12]. A *common cluster* is a taxon set that forms a pendant subtree in every input tree. Each cluster is solved independently. To account for blocks in the solution that might cross a cluster boundary, each cluster is solved twice: once on the cluster itself and once after attaching an artificial root taxon ρ above the cluster root, following the standard treatment of cluster reduction [3].

4 Column Generation

Let $\beta(i, v) \geq 0$ denote the negated dual price of the packing constraint (2) for tree i and internal vertex v at the current RMP optimum. The reduced cost of a candidate column S is negative (i.e. S can improve the RMP) exactly when

$$\sigma(S) \;:=\; |S| - \sum_{i=1}^{t} \sum_{v \in \mathrm{St}(T_i, S)} \beta(i, v) \;>\; 1. \tag{4}$$

The *pricing subproblem* is therefore to find valid blocks S maximizing $\sigma(S)$. When all prices vanish ($\beta(i, v) = 0$), constraint (4) reduces to maximizing $|S|$ subject to S being a valid block, which is precisely the classical *Maximum Agreement Subtree* (MAST) problem [4, 16]. The latter asks to find the largest subset of taxa on which all input trees induce the same rooted topology. We generalize MAST to a *weighted* objective σ (and abbreviate this generalization as MWAST), in which each internal vertex carries a weight that penalizes all blocks that contain this vertex in their Steiner tree. A consequence of branching (Section 5) is that the effective vertex weight $-\beta(i, v)$ is shifted by the duals of any branching rows acting on (i, v) and may become *positive*; the pricing objective and the algorithm that solves it (Section 7) must therefore handle vertex weights of arbitrary sign, unlike a pure cardinality MAST. Each column-generation iteration solves the pricing problem and computes an agreement subtree for each internal vertex v of an input tree of maximum weight where this vertex v is forced to be the root. Since one of these vertices is the root of the MWAST, the returned subtrees contain the MWAST. Importantly, when the subproblem solver returns no block S with $\sigma(S) > 1$, no column with negative reduced price exists and the RMP solution is optimal for the whole MP and this is where the column generation loop stops. To solve the subproblem, we adapt the cubic time dynamic programming algorithm of [7] that works on unit taxa

weights to arbitrary internal weights. In the special case of two input trees, we adopt the $O(n \log n)$ dynamic programming algorithm of [4].

5 Branching

As usual in set packing formulations, we branch on packing constraints, either setting the involved sum to zero or one. In particular, for a chosen tree i and internal vertex $v \in \mathrm{St}(T_i, X)$ the two children impose

$$\sum_{S \in \mathcal{S}: v \in \mathrm{St}(T_i, S)} a_S \leq 0 \qquad \text{and} \qquad \sum_{S \in \mathcal{S}: v \in \mathrm{St}(T_i, S)} a_S \geq 1.$$

In the first branch, the Steiner tree of no block is allowed to cross the vertex v and in the second branch, the Steiner tree of exactly one block has to cross vertex v.

This disjunction is exhaustive and partitions the solution space, so branch-and-price remains exact. The left hand side of both branching constraints is the same as the left hand side of (2) with identical coefficients for each column. Therefore, it impacts the pricing subproblem by simply adding the branching constraint dual to the corresponding internal weight. Note that this can lead to positive values for the internal weights. The branching vertex is selected by strong branching over the most fractional, most basis-splitting candidates: each candidate is scored by $4c(1-c)\sqrt{(1+r)(1+r')}$, where c is the LP mass covering (i, v) and r, r' split the basic columns of the current LP solution across the two children; the top candidates are then probed by a one-step LP look-ahead and the vertex maximizing the product of the two child objective increases is chosen. Nodes of the branch-and-bound tree are explored best-bound first, and any node whose LP bound can no longer beat the incumbent is pruned; a node whose master becomes infeasible under its branching constraints is pruned as admitting no forest.

6 Cutting Planes

We strengthen the LP with additional *clique cuts* [15]: for a set Q of pairwise conflicting columns, $\sum_{S \in Q} a_S \leq 1$ is valid. Violated cliques are separated from the fractional LP solution by a greedy clique separator run over a lexicographic column order. The separator is based on the clique separator of the solver HiGHS [10]. Cut separation is enabled only for $t \geq 3$, where it tightens the root bound.

Following [6] an active cut with dual penalty $\pi \leq 0$ must lower the reduced cost of any newly priced column that conflicts with *all* of its members (so that the master would give that column coefficient one in the cut). The pricing solver of Section 7 accounts for this by activating a cut's penalty precisely when the growing block conflicts with every member of the cut.

7 Branch and Bound Subproblem solver

The pricing problem (4) is solved by one of three algorithms. When no cuts are active, a dynamic program computes, for every inner vertex of every input tree, the best weighted agreement subtree rooted that vertex. For $t = 2$ this is an $O(n \log n)$ centroid-decomposition based algorithm [4] (the only variant that scales to the largest heuristic and lower-bound instances); a plain $O(n^2)$ two-tree dynamic program is available as a reference. For $t \geq 3$, a separate cubic time dynamic programming algorithm [7] also computes, for every program

is generalized from two to t trees. Both algorithms are adapted from the literature to also consider inner vertex weights.

A clique cut that is active in the LP and has a non-zero dual value, contributes its dual value to columns of the MP that enlarge the clique. This changes the objective function of the pricing subproblem in a way that the dynamic programming algorithms above cannot handle any more. Therefore, when clique cuts are present, pricing switches to a depth-first branch-and-bound over taxon insertions. Fixing a pair of taxa $\{x, y\}$ (for each $x, y \in X$), the search grows an agreement subtree by inserting further taxa at those insertion points that keep the induced topology consistent across all trees. Along the way it adds a cut penalty as soon as it is inevitable that any extension of the partial column will enlarge the cut. Each pair is seeded with an admissible upper bound precomputed by a per-pair dynamic program, and every node is pruned as soon as its bound plus the accumulated cut penalty cannot exceed the current incumbent. Taxa are considered in a fixed rank order for symmetry breaking.

8 Extension to a heuristic solver

The heuristic track of PACE 2026 handles two-tree instances ($t = 2$) of up to tens of thousands of taxa within a five-minute budget. On instances of this size the reduction and cluster-reduction rules can require a substantial amount of time. In this setting, we implemented them on top of Day's linear-time algorithm for comparing leaf-labeled trees [5], they run in $O(tn)$ time and therefore enjoy a time advantage over the their more straightforward (however easier verifiable) companion-implementations for the exact solver. A valid forest is kept ready at all times (initially the all-singletons forest) and is emitted as soon as the time budget is exhausted, so the solver can be stopped at any moment and still return a valid, non-empty answer.

The heuristic solver relies on the same machinery as the exact solver but recombines its ingredients in a more runtime- and memory-friendly manner to the price of suboptimality.

The overall pipeline is as follows. From the two input trees, an initial round of maximum-agreement-subtree extraction and greedy disjoint packing yields a first non-trivial valid incumbent. The instance is then repeatedly reduced (the cherry, chain, and near-cherry rules) and split into its common clusters, and each resulting block is solved by a column-pool matheuristic, while a sufficiently small irreducible core is instead handed to the exact solver of Section 3. This reduce/decompose/solve pass is restarted with fresh random seeds until the budget is spent, and the best valid forest found is returned. We describe the individual steps next.

Initial incumbent. Before any expensive step, we repeatedly compute maximum agreement subtrees of the remaining taxa, collect them as candidate blocks, and remove the largest one to make progress; this time-bounded loop yields a pool of large agreement subtrees, and a greedy disjoint packing over that pool is published immediately as a first non-trivial incumbent. This guarantees a useful answer even when the solver is stopped very early on a huge instance.

Reduce, decompose, solve per block. The instance is then kernelized by the reduction rules and split into its common clusters (Section 3); a sufficiently small irreducible core is solved to optimality by the exact solver of Section 3, its improving incumbents registered live. Each remaining block is handed to a column-pool matheuristic: a pool of candidate blocks is assembled from this MAST extraction and from the root column generation of Section 4 (optionally enriched with further dual-priced columns), and a greedy disjoint packing turns

it into a first valid forest for the block.

Feasibility pump. The block forest is improved by an LP-guided *feasibility pump* [8]: the pool LP is solved, its (slightly jittered) fractional column values weight a disjoint repair into a valid forest, the columns just used are penalized, and the perturbed LP is re-solved; successive rounds thus explore different LP roundings while every intermediate forest stays feasible by construction. The best forest is kept, and the pump stops once it reaches the pool LP lower bound or stalls.

Randomized restarts. The whole reduce/decompose/solve pass is repeated with fresh random seeds and varied tie-breaking until the budget is spent, keeping the best forest seen. Every block solution and every recombined forest is validity-guarded—any candidate that is not a valid agreement forest is rejected in favour of the current incumbent—so the published forest is always feasible.

9 Extension to a solver for the lower bound track

On this track each instance additionally specifies a pair (a, b), and any valid forest of size at most $\lfloor ak^* \rfloor + b$ is accepted. `woodcutter` chooses its strategy from how tight this guarantee is, at the threshold $a \leq 1.05$.[1]

Tight guarantee ($a \leq 1.05$). The margin presumably leaves room for little but a near-exact answer, so `woodcutter` simply runs the exact solver of Section 3 except to the difference that the solution process is interrupted and the incumbent is returned, whenever the requested approximation guarantee is already met.

Loose guarantee ($a > 1.05$). Computing k^* exactly is now wasteful. `woodcutter` instead first computes a *cheap dual bound*: root column generation *without branching* is run to LP optimality, and its converged value is a valid lower bound $L \leq k^*$ (Section 2). (Only a *converged* generation is used—a truncated one would over-estimate the LP—and otherwise the trivial $L = 1$ is kept, so $L \leq k^*$ always holds.) It then drives the primal side down as quickly as possible with the heuristic upper-bound machinery of Section 8 and outputs the current forest F the moment $|F| \leq \lfloor aL \rfloor + b$, i.e. as soon as the instance-specific guarantee is provably met (which implies $|F| \leq \lfloor ak^* \rfloor + b$ since $L \leq k^*$). Only if this does not certify in time does it fall back to tightening the distance between known primal and dual bound with the exact branch-and-price with the same earlier-termination criterion as in the case $a \leq 1.05$.

10 Use of Generative AI

OpenAI Codex (GPT 5.5) and Anthropic Claude Opus (versions 4.7 and 4.8) were used to assist (i) in implementing both previously published methodology for the MAF problem and the authors' own approaches, and (ii) in writing and editing the text of the present document and creating its illustrations. All AI-generated content, code and text alike, was critically reviewed and, where necessary, modified by the authors before inclusion.

References

1 Benjamin L. Allen and Mike Steel. Subtree transfer operations and their induced metrics on evolutionary trees. *Annals of Combinatorics*, 5(1):1–15, 2001.

[1] Throughout, the linear-time reductions and cluster decomposition of Section 8 are used in place of the exact solver's $O(n^2)$ preprocessing, which would time out at this scale.

2 Cynthia Barnhart, Ellis L. Johnson, George L. Nemhauser, Martin W. P. Savelsbergh, and Pamela H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

3 Magnus Bordewich and Charles Semple. On the computational complexity of the rooted subtree prune and regraft distance. *Annals of Combinatorics*, 8(4):409–423, 2004.

4 Richard Cole, Martin Farach-Colton, Ramesh Hariharan, Teresa Przytycka, and Mikkel Thorup. An $O(n \log n)$ algorithm for the maximum agreement subtree problem for binary trees. *SIAM Journal on Computing*, 30(5):1385–1404, 2000.

5 William H. E. Day. Optimal algorithms for comparing trees with labeled leaves. *Journal of Classification*, 2(1):7–28, 1985.

6 Jacques Desrosiers, Marco E. Lübbecke, Guy Desaulniers, and Jean Bertrand Gauthier. *Branch-and-Price*. Springer, 2024.

7 Martin Farach, Teresa M. Przytycka, and Mikkel Thorup. On the agreement of many trees. *Information Processing Letters*, 55(6):297–301, 1995.

8 M. Fischetti, F. W. Glover, and A. Lodi. The feasibility pump. *Mathematical Programming*, 104(1):91–104, 2005. doi:10.1007/S10107-004-0570-3.

9 Martin Frohn, Steven Kelk, and Simona Vychytilova. A branch-&-price approach to the unrooted maximum agreement forest problem. *Operations Research Letters*, 63:107364, 2025.

10 Qi Huangfu and Julian A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

11 Steven Kelk, Simone Linz, and Ruben Meuwese. Cyclic generators and an improved linear kernel for the rooted subtree prune and regraft distance. *Information Processing Letters*, 180:106336, 2023.

12 Simone Linz and Charles Semple. A cluster reduction for computing the subtree distance between phylogenies. *Annals of Combinatorics*, 15(3):465–484, 2011.

13 Marco E. Lübbecke and Jacques Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2005.

14 Neil Olver, Frans Schalekamp, Suzanne van der Ster, Leen Stougie, and Anke van Zuylen. A duality based 2-approximation algorithm for maximum agreement forest. *Mathematical Programming*, 198(1):811–853, 2023.

15 Manfred W. Padberg. On the facial structure of set packing polyhedra. *Mathematical Programming*, 5(1):199–215, 1973.

16 Mike Steel and Tandy Warnow. Kaikoura tree theorems: Computing the maximum agreement subtree. *Information Processing Letters*, 48(2):77–82, 1993.